

Práctica 3-DSA

Alumnos:

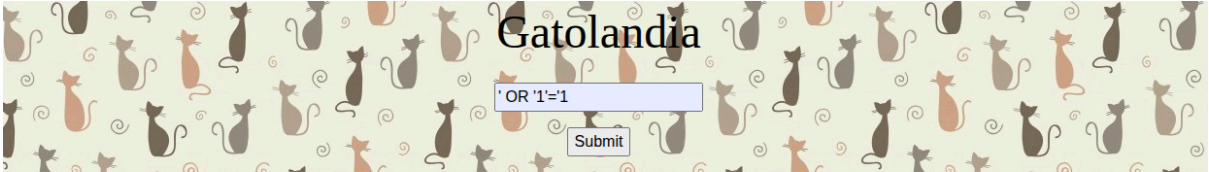
- Alvarez Jeremías Martín
- Palacios Tomás Agustín

Ejercicio Explotación y fix

Caso 1-Gatolandia

Con el objetivo de explorar la vulnerabilidad de **SQL Injection** ingresamos al reto **python_sqli** (<http://localhost:13001>)

En dicha página encontramos que, al ingresar un **id** del **1** al **10**, se muestran los datos correspondientes a ese usuario especificado pero al ingresar **' OR '1'='1** el sistema permitió el acceso sin credenciales válidas mostrando la información de todos los usuarios.



Gatolandia				
Id	Nombre	Personalidad	Nivel	Foto
1	Snowball	Mellow	80	
2	Smokey	Hot and Cold	140	
3	Pumpkin	Spacey	90	
4	Tabitha	Leisurely	40	
5	Bandit	Wild at Heart	180	

4	Tabitha	Leisurely	40	
5	Bandit	Wild at Heart	180	
6	Pepper	Shy	165	
7	Patches	Jealous	80	
8	Princess	Ditzy	125	
9	Ginger	Bashful	91	
10	Peaches	Capricious	45	
16161	Wally	flag{Bieeen_esta_eslaflag}	250	

SOLUCIÓN

Al explorar el código original identificamos la parte donde se permite la vulnerabilidad.

```
def get_cat_by_id(self, idGato):
    connection = 'mysql+mysqlconnector://%s:%s@%s:%s/%s' % (self.db_user, self.db_password, self.db_host, self.db_port, self.db_name)

    engine = create_engine(connection)
    connection = engine.connect()
    session = self.get_session()
    sql = "SELECT * FROM catbook WHERE id = '" + (str(idGato)) + "';"
    result=connection.execute(sql).fetchall()
```

Esto es **SQL Injection** porque concatena directamente el **idGato** en la consulta.

Lo modificamos de la siguiente manera para permitir una consulta parametrizada evitando que el **input** se ejecute como **SQL**:

```
query = text("SELECT * FROM catbook WHERE id = :id")
result = connection.execute(query, {"id": idGato}).fetchall()
```

¿Cómo funciona?

- **text(...)**: crea un objeto de consulta SQL parametrizada.
- Dentro de la cadena, **:id** es un **placeholder** (marcador de posición).
- No se sustituye directamente con concatenación de strings, sino que **SQLAlchemy** lo trata como un parámetro seguro. Esto evita que el valor de **idGato** se interprete como parte del SQL. Si alguien intenta poner **1 OR 1=1**, se pasa como texto literal, no como instrucción SQL.
- **connection.execute(...)**: ejecuta la consulta en la base de datos.
- El segundo argumento **{"id": idGato}** es un diccionario que le dice a **SQLAlchemy** "el parámetro **:id** vale **idGato**". Si alguien pone **idGato = "1 OR 1=1"**, la base lo interpreta como un **string literal**, no como SQL, y no devuelve todos los registros.

Ahora en la página al probar con un id válido sigue mostrando la información de ese usuario en particular pero al ingresar **' OR '1'='1'** ya no muestra toda la información de todos los usuarios.

Prueba realizada con id= 1:

Id	Nombre	Personalidad	Nivel	Foto
1	Snowball	Mellow	80	

Prueba realizada con ' OR '1'='1':

Id	Nombre	Personalidad	Nivel	Foto
				

Caso 2-Chequeador de Ping

En este caso al explorar el reto **python_rce** (<http://localhost:13002>) nos dimos cuenta que, al ingresar en el input una dirección IP como por ejemplo **127.0.0.1**, la página nos devuelve el ping correctamente:

Chequeador de Ping

Ingresá una IP para que nosotros le hagamos ping :)

Resultado:

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: seq=0 ttl=64 time=0.064 ms
```

```
--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 0.064/0.064/0.064 ms
```

Pero al concatenar algo malicioso como (**127.0.0.1; ls**), el servidor ejecuta **dos comandos**: primero **ping**, después **ls**, listando de esta manera los archivos.

Chequeador de Ping

Ingresá una IP para que nosotros le hagamos ping :)

Resultado:

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: seq=0 ttl=64 time=0.070 ms
```

```
--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 0.070/0.070/0.070 ms
```

```
bin
dev
entrypoint.sh
etc
home
lib
media
mnt
proc
root
run
sbin
```

En este caso probamos con el payload (**&& whoami**) para ver si nos devolvía el usuario con el que corre el proceso y el resultado fue el siguiente:

Chequeador de Ping

Ingresá una IP para que nosotros le hagamos ping :)

Resultado:

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: seq=0 ttl=64 time=0.054 ms
```

```
--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 0.054/0.054/0.054 ms
root
```

Ahora al intentar con (**|| echo vulnerable**) la respuesta fue:

Chequeador de Ping

Ingresá una IP para que nosotros le hagamos ping :)

Resultado:

vulnerable

Estos chequeos demuestran que estamos ante una vulnerabilidad **RCE** donde el atacante puede inyectar comandos arbitrarios y hacer que el servidor los ejecute.

SOLUCIÓN

En el código encontramos lo siguiente:

```
@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method=="POST" and "ip" in request.form:
        ip = request.form['ip']
        command = "ping -c1 " + ip
        try:
            check_ping= subprocess.check_output(command, shell=True).decode("utf-8")
        except subprocess.CalledProcessError as e:
            check_ping="El host con la IP ingresada no responde"
        return render_template('index.html', result=check_ping)
    return render_template('index.html')
```

De esta manera:

- El input del usuario (**ip**) se concatena directamente en un **string de comando**.
- Como se usa **shell=True**, cualquier cosa que escribamos en el campo IP se ejecuta en la shell.

La solución es **no concatenar el input y no usar shell=True**.

```
@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method=="POST" and "ip" in request.form:
        ip = request.form['ip']
        #Validar que sea una IP(solo numeros y puntos)
        if not re.match(r'^[0-9.]+$', ip):
            return render_template('index.html', result="IP inválida")
        try:
            check_ping= subprocess.check_output(["ping", "-c1", ip], text=True)
        except subprocess.CalledProcessError:
            check_ping="El host con la IP ingresada no responde"
        return render_template('index.html', result=check_ping)
    return render_template('index.html')
```

Ahora, el **fix** elimina la ejecución arbitraria de comandos al validar la entrada y usar **subprocess** de forma segura.

Prueba realizada con **127.0.0.1**:

Chequeador de Ping

Ingresá una IP para que nosotros le hagamos ping :)

Resultado:

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: seq=0 ttl=64 time=0.055 ms
```

```
--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 0.055/0.055/0.055 ms
```

Prueba realizada con **127.0.0.1;ls**:

Chequeador de Ping

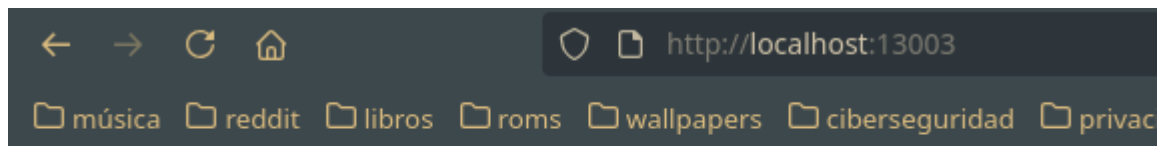
Ingresá una IP para que nosotros le hagamos ping :)

Resultado:

IP inválida

De esta manera podemos observar que, al ingresar una **ip** válida el resultado sigue siendo el esperado, pero al querer concatenar código malicioso esto ya no es permitido.

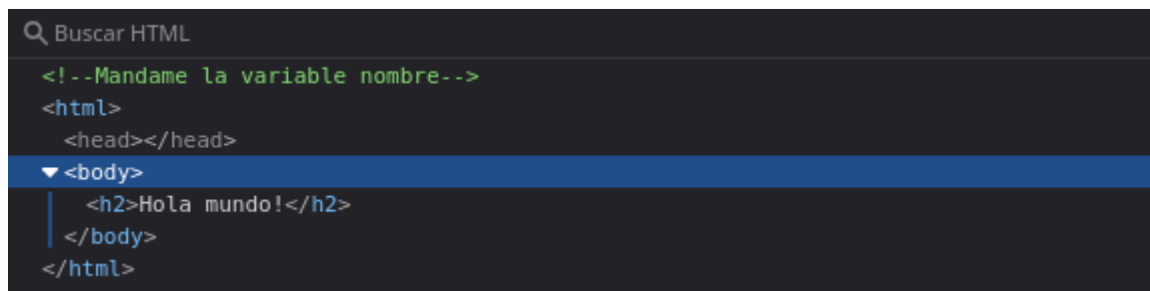
RETO 3 - python_ssti



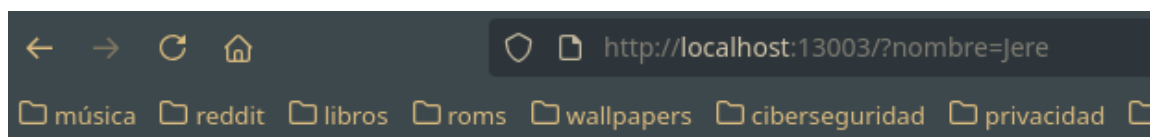
Hola mundo!

Al ingresar a la **URL** podemos ver en **HTML** el mensaje “**Hola mundo!**” sin ningún otro dato relevante en pantalla.

Al inspeccionar el código podemos ver el siguiente comentario: “**Mandame la variable nombre**”.



Ahora probemos mandarle un texto vía **URL**:



Hola Jere!

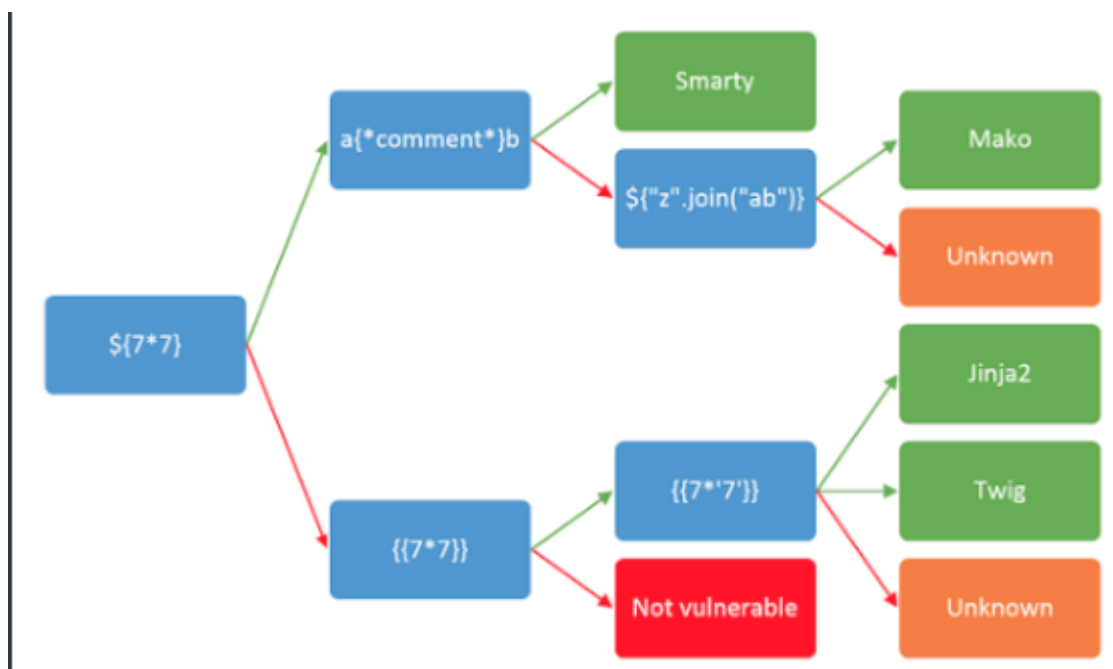
Como podemos ver, lo que mandemos como parámetro nombre, lo va a imprimir en pantalla.

Ahora bien, repasemos la definición de esta vulnerabilidad tomando como referencia lo visto en la teoría y expandiendo un poco más el concepto:

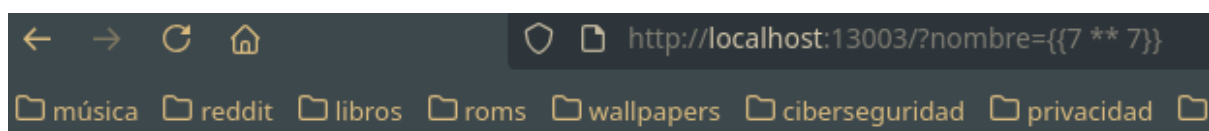
“**Server-Side Template Injection (SSTI)** vulnerabilities refer to weaknesses in web applications which attackers can exploit to inject malicious code into server-side templates. This allows them to execute arbitrary commands on the server, potentially leading to unauthorized data access, server compromise, or exploitation of additional vulnerabilities.”

Información obtenida de:

“<https://research.checkpoint.com/2024/server-side-template-injection-transforming-web-applications-from-assets-to-liabilities/>”.



Entonces, dado que lo que pasamos como parámetro es **procesado** y luego **renderizado** (como funciona por ejemplo **Django** al utilizar variables y luego mostrarlas en pantalla, mismo **Flask**), si respetamos la sintaxis de **Jinja2** (“`{{ }}`”), podemos lograr que se interprete nuestra variable como parte del código.



Hola 823543!

Como podemos ver, el código es interpretado. ¿Esto por qué pasa?

Si vemos el código fuente (**app.py**):

```
person['name'] = request.args.get('nombre')
template = '''<!--Mandame la variable nombre--><h2>Hola %s!</h2>''' % person['name']
return render_template_string(template, person=person)
```

Tenemos que lo que se mande por parámetro en la **URL** se pasa a la función **render_template_string** que no va a tratar lo que yo envié como un simple string sino que va a interpretar la sintaxis.

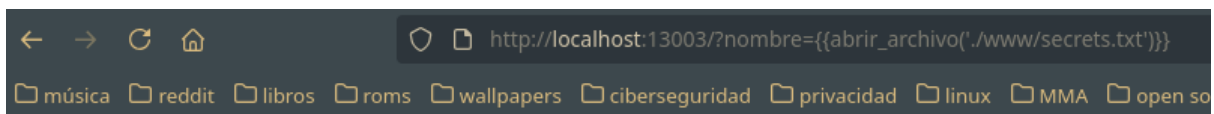
A este problema sumemosle el siguiente fragmento de código:

```
def abrir_archivo(f_name):
    with open(f_name) as f:
        return f.readlines()

app.jinja_env.globals['abrir_archivo'] = abrir_archivo
```

La función global “**abrir_archivo**” abre lo que le pasemos como parámetro, permitiéndonos leer (como atacantes) archivos propios del **server** y mostrarlos en pantalla.

Veamos una prueba:

A screenshot of a web browser window. The address bar shows the URL 'http://localhost:13003/?nombre={{abrir_archivo('./www/secrets.txt')}}'. Below the address bar, there is a navigation bar with several folder icons and labels: 'música', 'reddit', 'libros', 'roms', 'wallpapers', 'ciberseguridad', 'privacidad', 'linux', 'MMA', and 'open so'. The main content area of the browser displays the text 'Hola [flag{alto_hacker_sos!_firname_un_autografo}]!'.

Hola [flag{alto_hacker_sos!_firname_un_autografo}]!

Ahí pude acceder a un archivo alojado dentro del servidor mediante la función global “**abrir_archivo**”).

¿Cómo podemos solucionar esta vulnerabilidad?

Una posible solución sería modificar el código de la siguiente manera:

```
...
from flask import Flask, request, render_template_string

app = Flask(__name__)

@app.route('/')
def hello():
    person = {
        'name': request.args.get('nombre', 'mundo') # Si no se manda nada por parámetro 'nombre', se usa 'mundo' por defecto
    }

    template = '''
    <!-- Mandame la variable nombre -->
    <h2>Hola {{ person.name }}</h2>
    '''

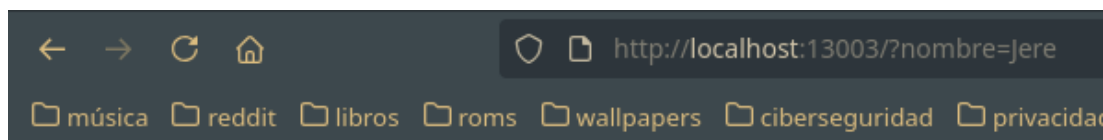
    return render_template_string(template, person=person)

if __name__ == "__main__":
    app.run(debug=False)
```

En primer lugar eliminar la función “**abrir_archivo**” ya que no debería exponer funciones sensibles (como esta que permite leer archivos) al entorno de las plantillas.

Luego como podemos ver en el código, el **template** ya no se construye concatenando e interpretando directamente el valor que recibimos por **URL**, sino que el parámetro recibido se va a interpretar **ÚNICAMENTE** como cadena de texto.

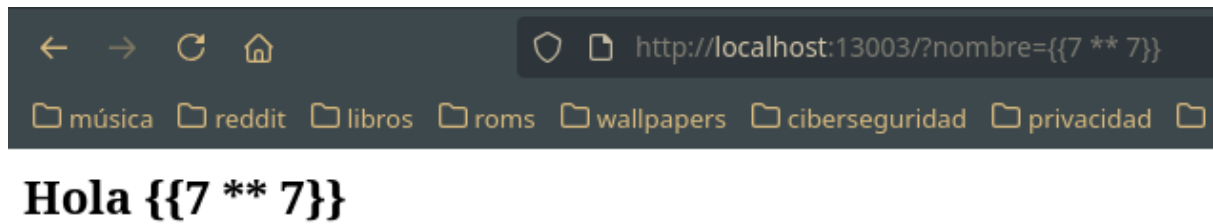
Entonces, si le mandamos un string normal:



Hola Jere

Funciona bien.

Si probamos enviarle código a interpretar en **Jinja2** como antes:



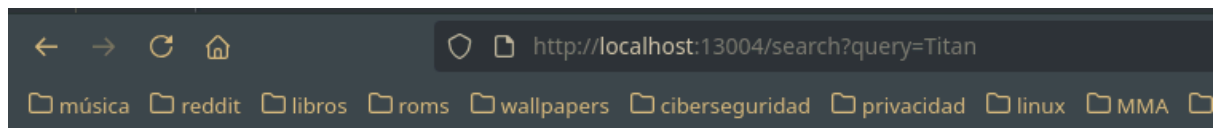
Sólo imprime en pantalla el **input interpretado en string**, no lo interpreta.
Ya la función de “**abrir_archivo**” se eliminó, por lo tanto no podemos mandarle una función entre llaves y acceder a los archivos del server.

Reto 4 - **python_xss**

En este reto nos encontramos con una página que nos permite buscar películas mediante un input:



Si ingresamos alguna cadena, nos van a aparecer sus respectivos resultados:

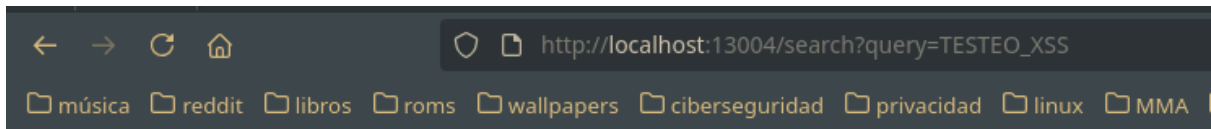


Resultados de búsqueda

Resultados para 'Titan':

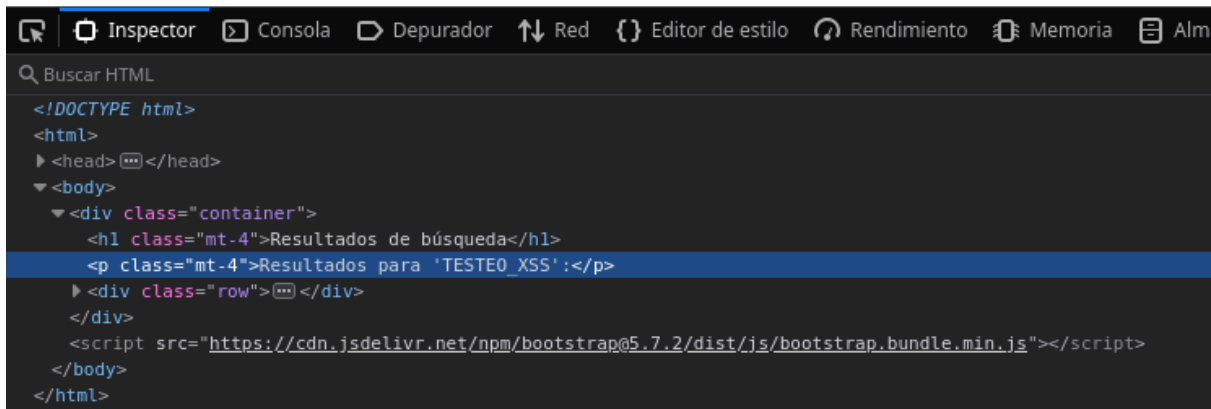


Si ingresamos cualquier otra cosa e inspeccionamos donde se almacena ese valor:

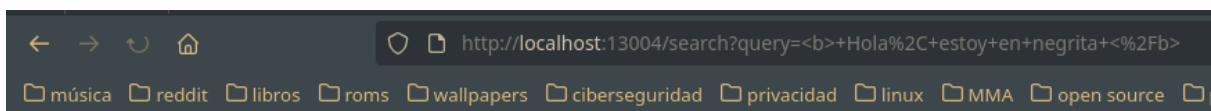


Resultados de búsqueda

Resultados para 'TESTEO_XSS':



Podemos ver que aparece como texto normal. Ahora bien... permite interpretar HTML, por ejemplo, si le paso en el input “ Hola, estoy en negrita ”:

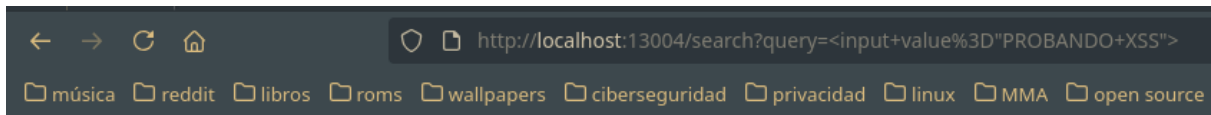


Resultados de búsqueda

Resultados para ' **Hola, estoy en negrita** ':

Efectivamente, está renderizando el HTML del usuario.

Mismo si le mando por input “<input value="PROBANDO XSS">”



Resultados de búsqueda

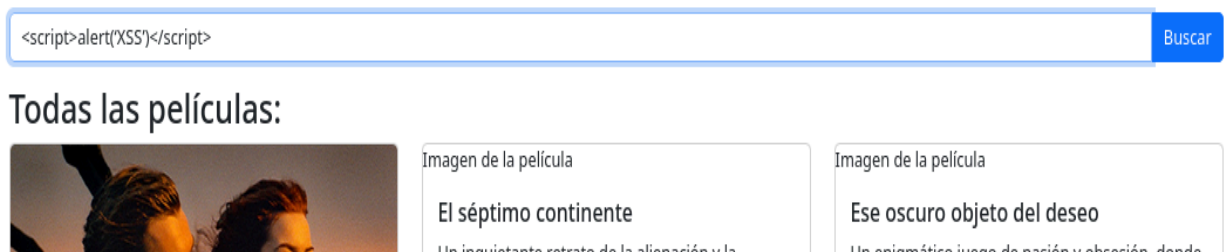
Resultados para "PROBANDO XSS":

Repasemos la definición vista en la teoría 8:

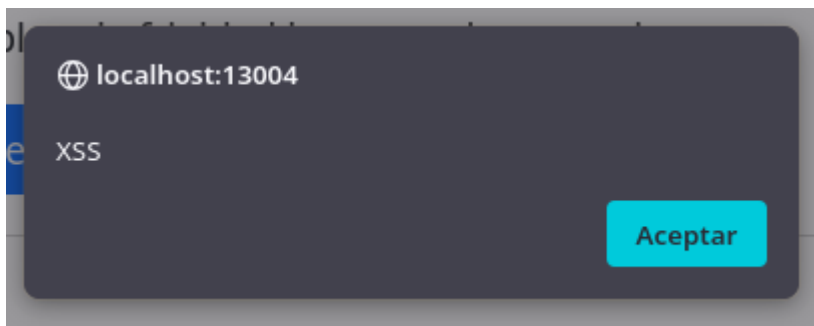
“Una vulnerabilidad de Cross Site Scripting (XSS) permite a un atacante ejecutar código client-side arbitrario en el navegador de un usuario. XSS ocurre cuando un input, no sanitizado por una aplicación, se incluye como parte del código de la misma y se ejecuta en el navegador. “

Por lo tanto, si hacemos:

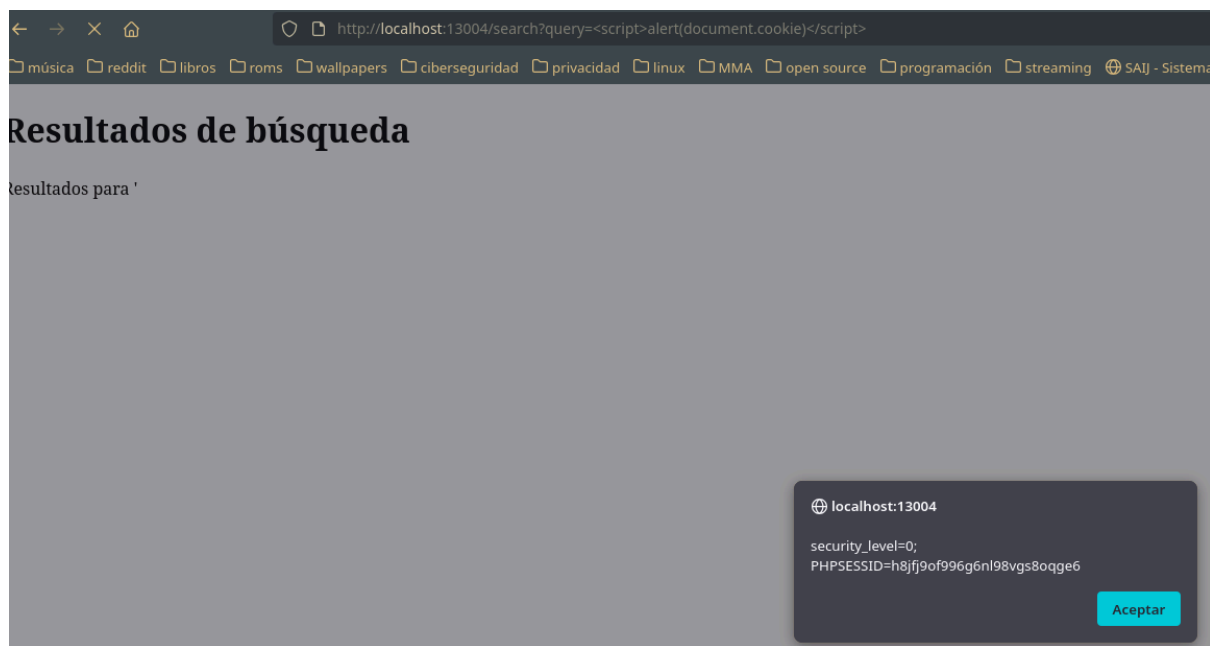
Buscador de películas



Nos muestra en pantalla la alerta:



También podemos ver las cookies:



¿Cómo podemos solucionarlo?

A comparación del código anterior:

```
1 from flask import Flask, request, render_template
2
3 app = Flask(__name__)
4
5 # Base de datos ficticia de películas
6 movies = {
7     '1': {'title': 'Titanic', 'image': 'https://images-na.ssl-images-amazon.com/images/S/pv-target-images/4721ec145e5bca07fe6a3719a2990'},
8     '2': {'title': 'El séptimo continente', 'image': 'https://pics.filmaffinity.com/El_saeptimo_continente-231616411-mmed.jpg', 'description': 'El séptimo continente'},
9     '3': {'title': 'Ese oscuro objeto del deseo', 'image': 'https://pics.filmaffinity.com/Ese_oscuro_objeto_del_deseo-767734797-large.jpg', 'description': 'Ese oscuro objeto del deseo'},
10    '4': {'title': 'Bad Boy Bubby', 'image': 'https://pics.filmaffinity.com/Bad_Boy_Bubby-978690686-large.jpg', 'description': 'Un viaje'}
11 }
12
13 @app.route('/')
14 def index():
15     return render_template('index.html', movies=movies)
16
17 @app.route('/search')
18 def search():
19     query = request.args.get('query', '')
20     results = []
21     for key, value in movies.items():
22         if query.lower() in value['title'].lower():
23             results.append({'title': value['title'], 'image': value['image']})
24
25     return render_template('search.html', query=query, results=results)
26
27 # Miguel, hace 2 semanas • Subo TP3
28
29 if __name__ == '__main__':
30     app.run(debug=True)
```

```
2026-rm-rf-CTF > practica3 > python_xss > www > templates > search.html > html > body > div.container > div.row > div.col-md-4
miguel, hace 2 semanas | 1 autor (miguel)
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Resultados de búsqueda</title>
5   <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.7.2/dist/css/bootstrap.min.css">
6 </head>
7 <body>
8   <div class="container">
9     <h1 class="mt-4">Resultados de búsqueda</h1>
10    <p class="mt-4">Resultados para '{{ query | safe }}':</p>
11    <div class="row">
12      {% for result in results %}
13        <div class="col-md-4">
14          <div class="card mb-4">
15            
16            <div class="card-body">
17              <h5 class="card-title">{{ result.title }}</h5>
18              <p class="card-text">Descripción de la película</p>
19              <a href="#" class="btn btn-primary">Ver detalles</a>
20            </div>
21          </div>
22        </div>
23      {% endfor %}
24    </div>
25  </div>
26
27  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.7.2/dist/js/bootstrap.bundle.min.js"></script>
28 </body>
29 </html>
30
```

Para corregir la vulnerabilidad lo que hacemos principalmente es sacar el “ | safe” del template “search.html” que lo que hace es permitirle a Jinja que confie en el valor de la query y que no transforme el input para que sea un texto común, permitiendo de esta forma que se interprete como etiquetas HTML.

```
2026-rm-rf-CTF > practica3 > python_xss > www > templates > search.html > ...
...
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Resultados de búsqueda</title>
5   <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.7.2/dist/css/bootstrap.min.css">
6 </head>
7 <body>
8   <div class="container">
9     <h1 class="mt-4">Resultados de búsqueda</h1>
10    <p class="mt-4">Resultados para '{{ query }}':</p>
11    <div class="row">
12      {% for result in results %}
13        <div class="col-md-4">
14          <div class="card mb-4">
15            
16            <div class="card-body">
17              <h5 class="card-title">{{ result.title }}</h5>
18              <p class="card-text">Descripción de la película</p>
19              <a href="#" class="btn btn-primary">Ver detalles</a>
20            </div>
21          </div>
22        </div>
23      {% endfor %}
24    </div>
25  </div>
26
27  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.7.2/dist/js/bootstrap.bundle.min.js"></script>
28 </body>
29 </html>
30
miguel, hace 2 semanas • Subo TP3
```

Por último, siguiendo lo mencionado en la teoría de Ezequiel Moreno, agregamos la flag HTTPOnly ya que “Setear Flag HTTPOnly en las cookies no permite que se manipulen desde javascript y por tanto reduce el impacto de explotación.”. Además agregamos una whitelist de dominios de los cuales puede ejecutar scripts.

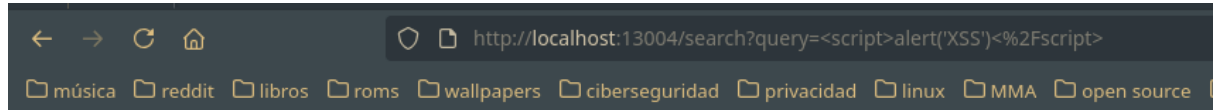
“Política de Seguridad del Contenido o (CSP) - del inglés Content Security Policy - es una capa de seguridad adicional que ayuda a prevenir y mitigar algunos tipos de ataque, incluyendo Cross Site Scripting (XSS) y ataques de inyección de datos. Un navegador compatible con CSP solo ejecutará scripts de los archivos fuentes especificados en una whitelist de dominios, ignorando completamente cualquier otro script (incluyendo los scripts inline y los atributos de HTML de manejo de eventos). Esta whitelist se define mediante la cabecera HTTP Content-Security-Policy.”

Y por último realizamos una limpieza de input que lo que hace es elimina espacios al inicio y al final, limita el largo del input y solo permite letras, números, espacios y guiones. Si alguien intenta mandar algo como <script>alert('XSS')</script>, el input se rechaza y queda vacío.

“INPUT VALIDATION: Una correcta validación y sanitización de inputs, constituye una primera línea de defensa contra datos no confiables.”

```
1  from flask import Flask, request, render_template
2  import re
3
4  app = Flask(__name__)
5
6  # Flag HTTPOnly para cookies de sesión como se recomienda en la teoría
7  app.config['SESSION_COOKIE_HTTPONLY'] = True
8
9  # Base de datos ficticia de películas
10 movies = {
11     '1': {'title': 'Titanic', 'image': 'https://images-na.ssl-images-amazon.com/images/S/pv-target-image'},
12     '2': {'title': 'El séptimo continente', 'image': 'https://pics.filmaffinity.com/El_saeptimo_continen'},
13     '3': {'title': 'Ese oscuro objeto del deseo', 'image': 'https://pics.filmaffinity.com/Ese_oscuro_obj'},
14     '4': {'title': 'Bad Boy Bubby', 'image': 'https://pics.filmaffinity.com/Bad_Boy_Bubby-978690686-larg'}
15 }
16
17 def limpiar_input(texto):
18     texto = texto.strip()
19     texto = texto[:50]
20
21     # Solo permite letras, números, espacios y guiones
22     if not re.match(r'^[a-zA-ZáéíóúÁÊÏÓÚñÑ0-9\s\-\_]+$', texto):
23         return ''
24
25     return texto
26
27
28 @app.after_request
29 def agregar_csp(response):
30     response.headers['Content-Security-Policy'] = (
31         "default-src 'self'; "
32         "script-src 'self' https://cdn.jsdelivr.net; "
33         "style-src 'self' https://cdn.jsdelivr.net; "
34         "img-src 'self' https:;"
35     )
36     return response
37
```

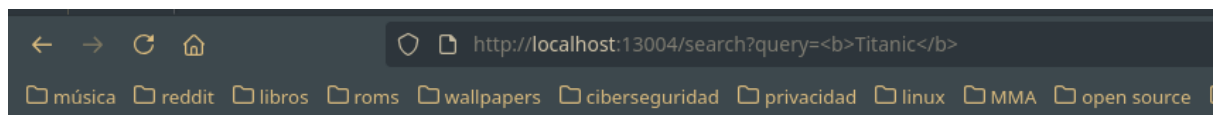
Problemos estas modificaciones:



Resultados de búsqueda

Resultados para "":

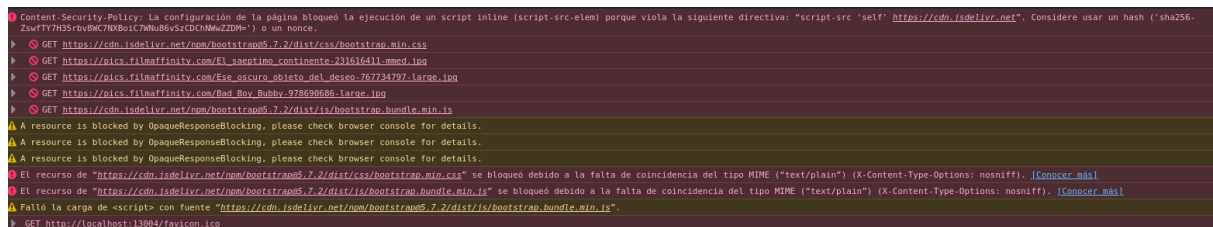
ya el `<script> alert ('XSS') </script>` no funciona, nos devuelve una cadena vacía. Mismo si ingresamos en el input lo que antes había funcionado, es decir "`<b> Titanic </b>`":



Resultados de búsqueda

Resultados para "":

Por último, para ver si nuestro CSP funcionó y bloqueó scripts, podemos ver en consola:



“Content-Security-Policy: La configuración de la página bloqueó la ejecución de un script inline...”